

PL-SQL-LAB1

Name: Keerthana Nellyeri

Roll no: AM.SC.U4AIE23120

1. Do all the questions discussed in the class

```
--Create a function to add 2 integer values and return sum
```

```
create or replace function add_two(a int, b int)
returns int as
$$
Declare
c int;
Begin
c:=a+b;
return c;
End;
$$
Language plpgsql;

select add_two(14,9)
```

	add_two integer
1	23

```
--Write a function that accepts an integer value and returns odd or even
```

```
create or replace function odd_even(a int)
returns varchar as
$$
Begin
if a%2=0 then
return 'even';
else
return 'odd';
end if;
End;
$$
Language plpgsql;

select odd_even(10)
select odd_even(7)
```

	odd_even character varying
1	even

	odd_even character varying
1	odd

```
--Consider the relation Emp(eno, ename, sal, dno)
--Write for which accepts an emp number and returns the salary of that employee
```

```
create table dept(
dno varchar(10) primary key,
dname char(10));

create table Emp(
eno varchar(10) primary key,
ename char(10),
sal numeric(10,2),
dept_no varchar(10) references dept(dno));

insert into dept values
('d1', 'Fashion'),
('d2', 'Medical');

insert into Emp values
('e1', 'Sree', '5000', 'd2'),
('e2', 'Farhan', '6700', 'd1');

create or replace function emp_sal(emp_no Emp.eno%type)
returns Emp.sal%type as
$$
Declare
salary Emp.sal%type;
Begin
select sal into salary from Emp where eno=emp_no;
return salary;
End;
$$
Language plpgsql;

select emp_sal('e2');
```

	emp_sal numeric 
1	6700.00

```
--Write a function which accepts a deptno and finds the no of employees in that department
```

```
create or replace function no_of_emp(department_no Emp.dept_no%type)
returns int as
$$
Declare
count_emp int;
Begin
select count(eno) into count_emp from Emp where dept_no=department_no;
return count_emp;
End;
$$
Language plpgsql;

select no_of_emp('d1');
```

	no_of_emp integer 
1	1

--Write a function which takes an employee no and give that employee 10% high in salary if average
 --salary of the corresponding department is greater than employee's current salary.

```
create or replace function hike_sal(emp_no emp.eno%type)
returns emp.sal%type as
$$
declare
    salary emp.sal%type;
    avg_salary emp.sal%type;
    dept emp.dept_no%type;
begin
    select sal into salary from emp where eno = emp_no;
    select dept_no into dept from emp where eno = emp_no;
    select avg(sal) into avg_salary from emp where dept_no = dept;

    if salary < avg_salary then
        update emp set sal = sal + (sal * 0.10) where eno = emp_no;
        select sal into salary from emp where eno = emp_no;
    end if;
    return salary;
end;
$$
language plpgsql;

select hike_sal('e1');
```

	hike_sal numeric
1	5000.00

--Write a function which accepts an employee name and returns the dept name of the employee

```
create or replace function dept_name(emp_name emp.ename%type)
returns dept.dname%type as
$$
declare
    dept_name dept.dname%type;
begin
    select d.dname into dept_name from emp e join dept d on e.dept_no = d.dno where e.ename = emp_name;
    return dept_name;
end;
$$
language plpgsql;

select dept_name('Sree');
```

	dept_name character
1	Medical

2. Consider the following schema . Emp(eno, ename, sal) EMP_PROJ(eno, pno, prjt_hrs).

a) Write a function ProjectLoad that returns the total project working hours for the given eno.

```

create or replace function projectload(emp_no emp.eno%type)
returns int as
$$
declare
    total_hrs int;
begin
    select sum(prjt_hrs) into total_hrs from emp_proj where eno = emp_no;
    return total_hrs;
end;
$$
language plpgsql;

```

b) Write a PL/SQL code to update the salary of an employee if the employee earn less than the average salary. New salary is current sal + difference between current sal and average salary

```

do
$$
declare
    avg_sal numeric(10,2);
    current_sal numeric(10,2);
begin
    select avg(sal) into avg_sal from emp;
    for r in (select eno, sal from emp) loop
        current_sal := r.sal;
        if current_sal < avg_sal then update emp set sal = current_sal + (avg_sal - current_sal) where eno = r.eno;
        end if;
    end loop;
end;
$$;

```

3. Consider the following tables Item(ino, iname, unit_price) Transaction(tr_no, ino, qty)
Write a function to accept an item no from the user. If transaction has been made for less than 2 times for that item(check from transaction table), delete the item from the Item table.

```

create table item(
    ino varchar(10) primary key,
    iname varchar(20),
    unit_price numeric(10,2)
);

create table transaction(
    tr_no varchar(10) primary key,
    ino varchar(10),
    qty int
);

insert into item values
('i1', 'pen', 10),
('i2', 'book', 40),
('i3', 'pencil', 5);

insert into transaction values
('t1', 'i1', 5),
('t2', 'i1', 2),
('t3', 'i2', 3);

```

```

create or replace function check_and_delete_item(item_no item.ino%type)
returns text as
$$
declare
    trans_count int;
begin
    select count(*) into trans_count from transaction where ino = item_no;
    if trans_count < 2 then delete from item where ino = item_no;
        return 'item deleted';
    else
        return 'item not deleted';
    end if;
end;
$$
language plpgsql;

select check_and_delete_item('i1');

```

Q4. Consider a Bank database which includes the following tables.

ACCOUNTS(ac_no,br_no, cust_no,ac_type,bal)

BRANCHES(br_no, br_name, loc)

CUSTOMER(cno, cname, c_type)

```

create table branches(
    br_no varchar(10) primary key,
    br_name varchar(50),
    loc varchar(50)
);

create table customer(
    cno varchar(10) primary key,
    cname varchar(50),
    c_type char(1)
);

create table accounts(
    ac_no varchar(10) primary key,
    br_no varchar(10) references branches(br_no),
    cust_no varchar(10) references customer(cno),
    ac_type varchar(20),
    bal numeric(12,2)
);

insert into branches values
('b1','main branch','city center'),
('b2','north branch','north zone');

insert into customer values
('c1','arun',null),
('c2','meera',null);

insert into accounts values
('a101','b1','c1','savings',8000),
('a102','b1','c1','current',3000),
('a201','b2','c2','savings',4000);

```

- a) Write a function that accepts a threshold value and a customer number . The program updates the c_type based on the threshold value. Ie. If balance> threshold then class A, else class B.

```

create or replace function update_customer_type(threshold numeric, cust_no customer.cno%type)
returns text as
$$
declare
    total_bal numeric;
begin
    select sum(bal) into total_bal from accounts where cust_no = cust_no;
    if total_bal > threshold then update customer set c_type = 'a' where cno = cust_no; return 'customer type updated to a!';
    else
        update customer set c_type = 'b' where cno = cust_no;
        return 'customer type updated to b!';
    end if;
end;
$$
language plpgsql;

```

b) Write a function called CloseBranch that takes two arguments (the branch to be closed and the branch to take over the accounts) and transfers all accounts at the closing branch to the new branch and removes the closing branch.

```

create or replace function closebranch(old_br_no branches.br_no%type, new_br_no branches.br_no%type)
returns text as
$$
begin
    update accounts set br_no = new_br_no where br_no = old_br_no;
    delete from branches where br_no = old_br_no;
    return 'branch closed and accounts transferred!';
end;
$$
language plpgsql;

```

c) Write a function that implements a "safe" withdrawal operation, that only permits a withdraw if there sufficient funds in the account to cover it.

```

create or replace function safe_withdraw(acno accounts.ac_no%type, amount numeric)
returns text as
$$
declare
    current_bal numeric;
begin
    select bal into current_bal
    from accounts
    where ac_no = acno;

    if current_bal >= amount then
        update accounts set bal = bal - amount where ac_no = acno;
        return 'withdrawal successful';
    else
        return 'insufficient funds';
    end if;
end;
$$
language plpgsql;

```